



ARTIFICIAL INTELLIGENCE

5 Proven Query Translation Techniques To Boost Your Performance

How to get near-perfect LLM performance even with imperfect inputs

Thuwarakesh Murallie

Aug 8, 2024 11 min read

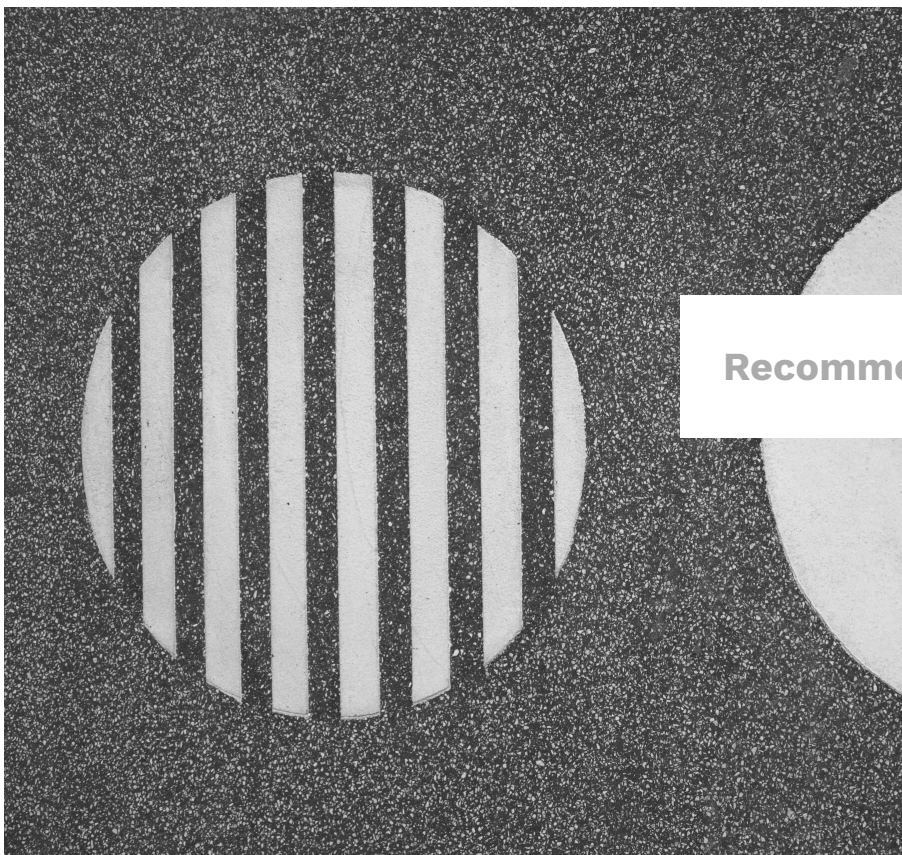
[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)[Recommended Article](#) [Close](#)

Photo by [travelnow.or.crylater](#) on [Unsplash](#)

You can't be more wrong than assuming the user provides the LLM the perfect questions. Rather than directly

if we refine the user's problem? That's query translation.

We built an app that lets users query through all the documents my company ever produced. These include PPTs, project proposals, progress updates, deliverables, documentation, etc. It was remarkable because many such attempts in the past fell short. Thanks to RAGs, this time, it was very promising.

We did a demo, and everyone was excited to use it. The rollout was for a small, selected batch of staff. The feedback noticed wasn't very exciting to us.

This was expected to be a game-changer in the way we worked. But most users tried the app only a few times and then quit. They quit the app as if it were a toy project.

The logs showed satisfactory results. However, the feedback from real users who used the app to determine the lessons we learned led us to think about *query translation techniques* to overcome ambiguity in user input.

Here's an example situation.

One user is interested in the fashion-related business. He advised one of our clients, "XYZ," to acquire. He asked, "What are the fashion-related acquisitions made by our top partners?" The app searched through our deliverables and came up with a list of a dozen companies. Yet, the list was different from what the user expected. XYZ partner had acquired (say) 7 fashion stores. But the list we provided was of 12 companies. The user, who is also a tester, is well aware of the acquisitions there have been.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Recommended Articles

No wonder people stopped using the tool. But thanks to the phased-out rollout technique, the lost trust is reversible.

We've made a series of changes to the app to fix this issue. One crucial update was query translations.

This post intends to introduce our different query translation techniques but not a deep dive. For instance, some of these techniques can be combined with prompting techniques like few-shot prompting and chain of thoughts to make a more powerful RAG. However, these techniques are for another post.

Advanced Recursive and Follow-Up Retrieval: Better RAGs

Let's explore the techniques one by one. But let's start with a basic RAG example.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Basic RAG Example

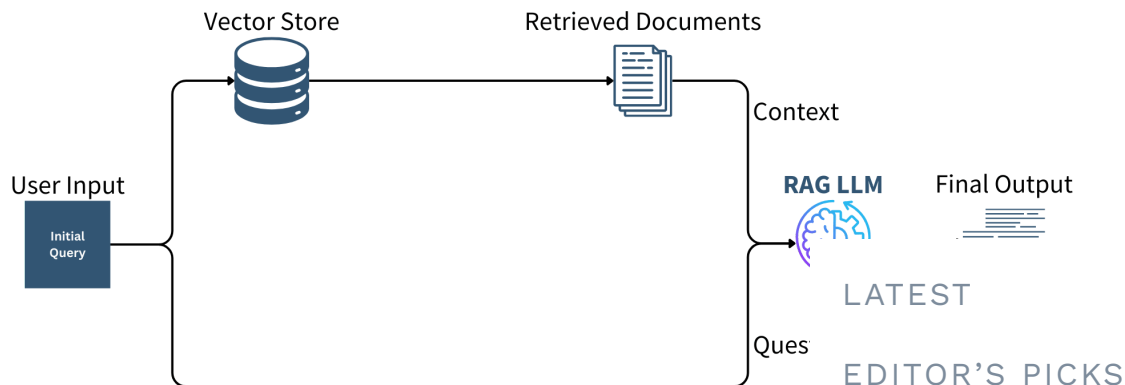
All RAG applications would have at least a vector store and a language model. The process is simple. Before the LLM answers the user's question, it searches a database for context or prior knowledge, it searches a database for context information and generates more accurate responses.

The following diagram illustrates the simplest

Recommended Articles

How Basic RAG Works?

The user's question is passed through the vector store to retrieve relevant documents. These documents are used as context for the LLM to answer the user's question.



Basic RAG application Workflow – Image by the au

In a simple RAG application, only one commur LLM model exists. This can be OpenAI GPT mo even your locally hosted model.

The following code implements the steps in th use this as our base to build on the other tech post.

```

# This is to securely load our sec
from dotenv import load_dotenv
load_dotenv()

# 1. Load the content
# -----
import bs4
from langchain_community.document_loaders import impc

loader = WebBaseLoader(
    web_paths=( "https://docs.djangoproject.com
    bs_kwargs=dict(
        parse_only=bs4.SoupStrainer(
            id="docs-content"
        )
    ),

```

Recommended Articles

RAG LLM Final Output
LATEST
EDITOR'S PICKS
DEEP DIVES
NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

```

)
doc_content = loader.load()

# 2. Indexing
# -----
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings

text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
docs = text_splitter.split_documents(doc_content)

vector_store = Chroma.from_documents(documents=docs)
retriever = vector_store.as_retriever()

# 3. LLM
# -----
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(temperature=0.5)

# 4. RAG Chain
# -----
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableChain
from langchain_core.output_parsers import StrOutputParser

prompt = """
Answer the question in the below context:
{context}

Question: {question}
"""

prompt_template = ChatPromptTemplate.from_template(prompt)

chain = (
    {"context": retriever, "question": RunnableChain(
        | prompt_template
        | llm
        | StrOutputParser()
    )

```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Recommended Articles

```
# 5. Invoking the chain
# -----
response = chain.invoke(
    "How can I improve site speed?",
)

print(response)
```

In the above code, we've used a web-based lookup to retrieve a page from Django's documentation page and store it in a Chroma vector store. Instead of the document retriever, you can play around with different web pages, local files, and many more.

Since we don't use a sophisticated retrieval technique, we passed the retriever straight to the final RAG chain. You can use another retriever chain instead of the retriever to apply various subsequent techniques. The rest of the article will show how to construct the retrieval chains.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

6 Python GUI Frameworks to Create Desktop and Mobile Apps.

Recommended Articles

Step-back prompting

Generate consistent answers that don't contain too much broader context.

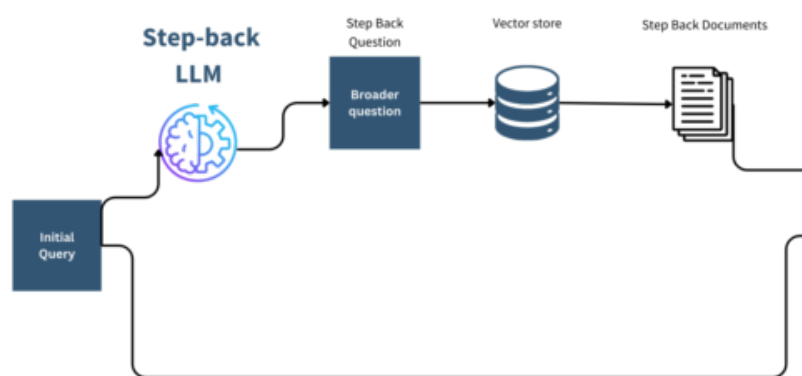
Step-back prompting is very similar to the basic prompting. When asking the user an initial question, we retrieve information from the database with a broader question.

The broader question captures more contextual information than the specific question. As a result, the final LLM can give the user more helpful information that doesn't contradict a broader context.

This is often useful when the initial queries are too specific and detailed but lacking the overall view.

How Step-back Prompting in RAG Works?

LLM generates a broader version of the user's input and retrieves relevant documents. These documents are used as context to answer the user's initial question.



thumarakesh.medium.com

Step-back prompting workflow – image by the author

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Recommended Articles

Here's the code implementation for step-back prompting that we use a different, where we passed a rephrased question to a basic RAG example.

```
# 4. RAG Chain
# -----
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableChain
from langchain_core.output_parsers import StrOutputParser

# 4.1 Step back prompting
step_back_prompt = """
You are an expert software engineer.
Your task is to rephrase the given question in a broader context.
```


Example 1

Question: How to improve Django performance?

Output: what factors impact web app performance?

Example 2

Question: How to optimize browser cache in Django?

Output: What are the different caching options?

Question: {question}

Output:

"""

LATEST

step_back_prompt_template = ChatPromptTemplate

EDITOR'S PICKS

retrieval_chain = step_back_prompt_template |

DEEP DIVES

4.2 RAG chain

prompt = """

Answer the question in the below context.

Your response should be comprehensive and not

If the context is not relevant to the question

{context}

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Question: {question}

"""

prompt_template = ChatPromptTemplate

Recommended Articles

rag_chain = (

 {"context": retrieval_chain, "question": F

 | prompt_template

 | llm

 | StrOutputParser()

)

Step-back prompting is helpful for application
broader context is critical. The LLM would pro
answers to related questions.

HyDE (Hypothetical Document Em

Generate contextually rich answers with relevant sources

HyDE is a recent and popular technique for document retrieval. The idea is to generate a document with the LLM's prior knowledge. Then, the document retrieves relevant context from the vector store.

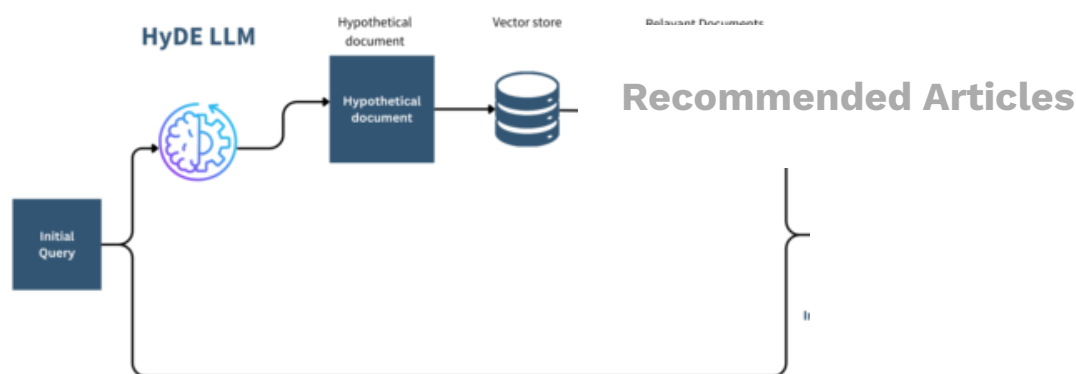
A good use case for HyDE is when the user often uses layman's terms to describe the problem, but the information store is very technical. Furthermore, the LLM's prompt provides more keywords to retrieve relevant information.

For instance, a query like 10 ways to improve LLM performance would provide an all-rounded answer covering the cost implications, caching, compression, etc.

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)

How HyDE in RAG Works?

The LLM creates a hypothetical document based on the user's input and uses it to retrieve relevant documents from the vector store. These documents are used as context to answer the user's query.

[Submit an Article](#)

thuwarakesh.medium.com

HyDE document retrieval process – image by the author

Here's the code implementation for the above process. At the same time, I've only provided the snippet that re-creates the RAG chain with HyDE.

4.1 HyDE prompting

```
hyde_prompt = """
```

You're an AI language assistant.

Your task is to generate a more broader version of the question b

By doing so, you're helping the user with more information.

Don't explain the question. Only provide a more broader version o

Question: {question}

Output:

```
"""
```

```
hyde_prompt_template = ChatPromptTemplate.from
```

[LATEST](#)

```
retrieval_chain = hyde_prompt_template | llm |
```

[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)

3 Ways to Deploy Machine Learning Models

[WRITE FOR TDS](#)

Multi-query

[Sign in](#)

Retrieve more documents overcoming the d
similarly search and get more relevant answ

[Submit an Article](#)

Multi-query is a technique that helps overcom
distance-based searches in vector
use cosine similarity to retrieve ve
method works well as long as the documents
reasonable similarities. However, the retrieval
short when distance-based similarity does not

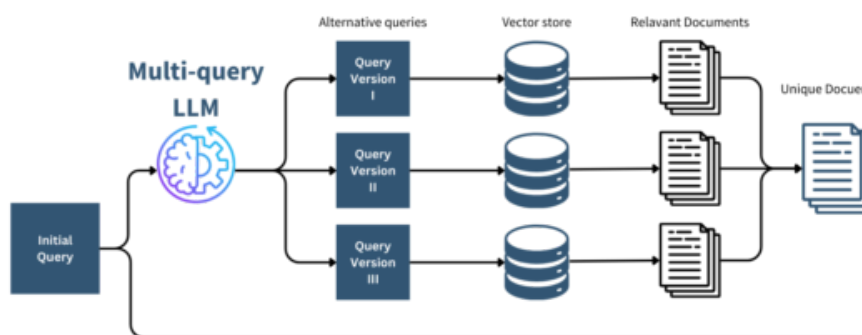
Recommended Articles

In the multi-query approach, we ask the LLM t
versions of the same query. For instance, a qu
speed up Django apps will be translated to an
"How to improve Django-based web apps' perf
queries together will retrieve more relevant do
vector store.

As an intermediary step, we must take a unique list of documents before we pass these documents to our final RAG-LLM. This is because there's a good chance that more than one query would retrieve the same documents. Passing them all with duplicates would reach the token threshold of LLM's for no good.

How Multi-query in RAG Works?

The user's question is passed through the Multi-query LLM to generate versions or retrieves documents from the vector store, and a unique set of documents will go to answer the user's question.



thuwarakesh.medium.com

Multi-query retrieval workflow – image by the aut

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

The code implementation also has **Recommended Articles** dedupes the document. The rest is ... methods.

```
from langchain.load import loads, dumps
```

```
def get_unique_documents(documents: list[list])
    # Flatten list of lists, and convert each
    flattened_docs = [dumps(doc) for sublist i
    # Get unique documents
    unique_docs = list(set(flattened_docs))
    # Return
    return [loads(doc) for doc in unique_docs]
```

```
# 4.1 Multi-query prompting
multi_query_prompt = """
```

You are an AI language model assistant.

Your task is to create five versions of the user's question to feed by offering multiple perspectives on the user's question, your goal is to give these alternative questions, each on a new line.

Question: {question}

Output:

"""

```
multi_query_prompt_template = ChatPromptTemplate.from_template(mu
```

```
retrieval_chain = (
    multi_query_prompt_template
    | llm
    | StrOutputParser()
    | (lambda x: x.split("\n"))
    | retriever.map()
    | get_unique_documents
)
```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

5 Python Decorators I Use in Almost All My Projects

Submit an Article

RAG-Fusion

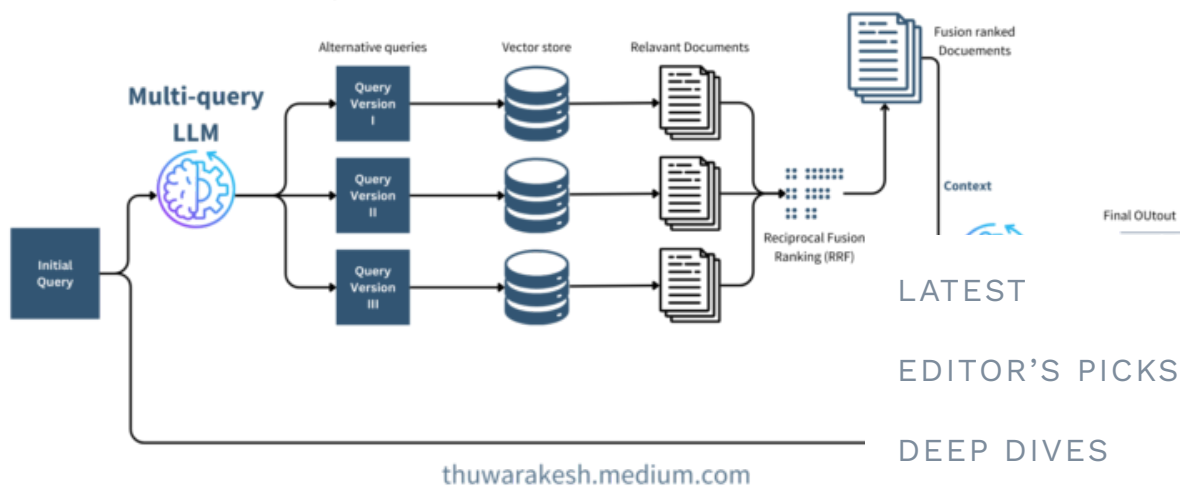
More relevant document plays a role in the final answer. **Recommended Articles**

RAG fusion is similar to multi-query on the document front. Once again, we ask the LLM to generate five versions of the initial query. We then retrieve documents based on these versions and combine them.

However, while combining and deduping the documents, we rank them according to their relevancy. Here's a representation of the RAG-fusion process.

How RAG-Fusion Works?

The user's question is passed through the Multi-query LLM to generate versions of the same. Each version retrieves documents from the vector store, and a **fusion-ranked** set of documents will go to the RAG LLM as context to answer the user's question.



RAG-fusion workflow – image by the author.

Instead of only deduping, we sort the documents by relevance. Reciprocal fusion ranking (RRF) is a clustering system that ranks documents based on their relevance to multiple queries. If a document appears in more than one query, RRF would rank it higher. If a document appears only in one of the queries, RRF would rank it lower. If a document is less similar, RRF would rank this document lower. RRF would rank the more relevant information and the less relevant information.

If more query versions retrieved the same document, then RRF would rank it higher. If a document appears only in one of the query versions, RRF would rank this document lower. If a document is less similar, RRF would rank this document lower. RRF would rank the more relevant information and the less relevant information.

```
def reciprocal_rank_fusion(results: list[list])
    """ Reciprocal_rank_fusion that takes multiple results and an optional parameter k used in the formula """
```

```
fused_scores = {}
```

```
for docs in results:
    for rank, doc in enumerate(docs):
        doc_str = dumps(doc)
        if doc_str not in fused_scores:
            fused_scores[doc_str] = 0
        previous_score = fused_scores[doc_str]
        fused_scores[doc_str] += 1 / (rank + 1)
```

```

reranked_results = [
    (loads(doc), score)
    for doc, score in sorted(fused_scores.items(), key=lambda
]

return reranked_results

# 4.1 RAG-fusion prompting
multi_query_prompt = """
You are an AI language assistant.
Your task is to generate 5 different versions
By doing so, you're helping the user to overcome
Provide these alternative questions separated
Question: {question}
Output:
"""

multi_query_prompt_template = ChatPromptTemplate(

retrieval_chain = (
    multi_query_prompt_template
    | llm
    | StrOutputParser()
    | (lambda x: x.split("\n"))
    | retriever.map()
    | reciprocal_rank_fusion
)

```

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Recommended Articles

This is How I Create Dazzling Dashboards Pi

Decomposition

When answering complex problems, let the
to pieces and construct the answer step-by

There are situations where it's better not to di
answering. An excellent approach to solving m

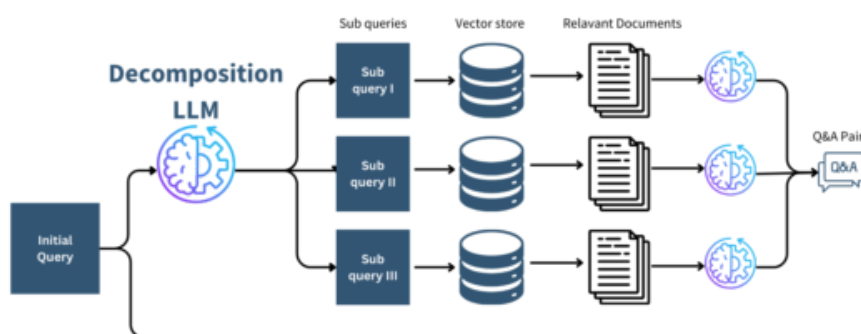
is to break the problem into pieces and answer them individually.

This isn't just an LLM technique, right?

Yes, we're trying to break the initial question into multiple sub-questions in query decomposition. Answering these sub-questions will provide bits and pieces to answer the initial query.

How Query decomposition in RAG Works?

The user's query is divided into sub-queries. Relevant documents for these queries are retrieved from the data store and answered individually. These Q&A pairs will be used as the context for the final query.



thuwarakesh.medium.com

Query decomposition in RAG

Recommended Articles

As you may see in the diagram, we retrieve relevant documents for each sub-questions and answer them separately. We then pass these question-and-answer pairs to the final LLM. Now the LLM has more granular information to solve the problem.

4.1 Decomposition prompting

```

decomposition_template = """You are an AI language model.
Your task is to break the following question into 5 sub-questions.
By doing so, you're helping the user construct a better prompt.
Provide these alternative questions separated by new lines.
Original question: {question}
Output:
  
```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

"""

```

decomposition_prompt_template = ChatPromptTemplate.from_template(

def query_and_combine(questions: list[str]) -> str:
    print(questions)
    qa_pairs = []
    for q in questions:
        r = basic_rag_chain.invoke(q)
        qa_pairs.append((q, r))

    qa_pairs_str = "\n".join([f"Q: {q}\nA: {a}"

    print(qa_pairs_str)

    return qa_pairs_str

retrieval_chain = (
    {"question": RunnablePassthrough()}
    | decomposition_prompt_template
    | llm
    | StrOutputParser()
    | (lambda x: x.split("\n"))
    | query_and_combine
)

```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Recommended Articles

Final thoughts

There are many steps from a demo app to pro
inevitable step is query translation.

The problems we solve differ in complexity. Th
querying needs to be taken into account. The
drawbacks should be addressed. These are a l

There is no single right way to pick the best q
technique. In real apps, you might have to con
techniques to get the desired output.

Thanks for reading, friend! Say Hi to me on [LinkedIn](#), [Twitter](#), and [Medium](#).

• • •

WRITTEN BY

Thuwarakesh Murallie

See all from Thuwarakesh Murallie

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

Artificial Intelligence

Hands On Tutorials

Mac

Python

Retrieval Augmented

WRITE FOR TDS

Sign in

Submit an Article

Share This Article

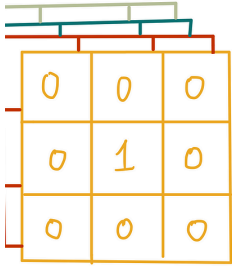


Towards Data Science is a community where you can share your insights to reach our global audience through the TDS Author Payment Program.

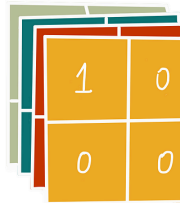
Recommended Articles

Write for TDS

Related Articles



max
pooling



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read



ARTIFICIAL INTELLIGENCE

How to Forecast Hierarchical Time Series

A beginner's guide to forecast reconciliation

Dr. Robert Kübler

August 20, 2024 13 min read



ARTIFICIAL INTELLIGENCE

What Do Large Language Models “Understand”?

A deep dive on the meaning of understanding and how it applies to LLMs

Tarik Dzekman

August 21, 2024 31 min read

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

DATA SCIENCE

Recommended Articles

Here's how to u
detect signals v
lines of...

Piero Paialunga

August 21, 2024



MACHINE LEARNING

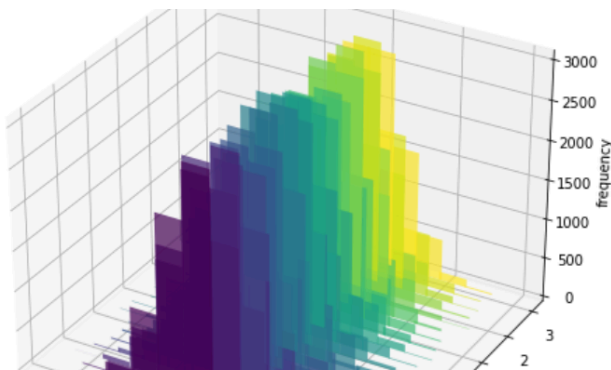
DATA SCIENCE

3 AI Use Cases (That Are Not a Chatbot)

Feature engineering, structuring unstructured data, and lead scoring

Shaw Talebi

August 21, 2024 7 min read



DATA SCIENCE

Must-Know in Statistics: The Bivariate Normal Projection Explained

Derivation and practical examples of this powerful concept

Luigi Battistoni

August 14, 2024 7 min read

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Recommended Articles



Your home for data science and AI. The world's leading publ
data analytics, data engineering, machine learning, and artifi
professionals.

© Insight Media Group, LLC 2026

[Subscribe to Our Newsletter](#)

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

[WRITE FOR TDS](#)

[Sign in](#)

[Submit an Article](#)

Recommended Articles